

Types de de données abstraits

La liste

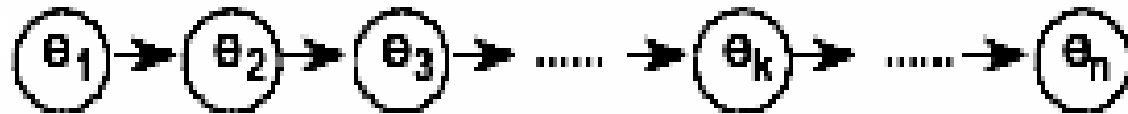
rouanehm@gmail.com

Notion de liste (1/2)

- Suite d'éléments $(e_1, e_2, e_3, \dots, e_n)$ de même type.
- Une liste est une suite finie, éventuellement vide, d'éléments repérés par leur rang dans la liste.
- Accès séquentiel $e_1 \rightarrow e_2 \rightarrow e_3 \rightarrow \dots \rightarrow e_n$
- Structure dont :
 - Les éléments peuvent être traités les uns à la suite des autres
 - Ajout ou retrait d'éléments n'importe où dans la liste
 - Accès à n'importe quel élément
- La liste est une généralisation de la pile (Stack) /file (Queue)

Notion de liste (2/2)

- Chaque élément de la liste est rangé à une certaine place.
- Les éléments d'une liste sont ordonnés en fonction de leur place.
- Le nombre total d'éléments, et par conséquent de places, est appelé longueur de la liste.
- Une liste vide est d'une longueur égale 0.



Opérations sur une ADT liste (1/2) :

Interface IList

- `add(e,index)` : ajoute l'élément `e` à la position `index`.
- `addFront(e)`: ajoute un élément au début de la liste.
 - Peut être définie avec `add(e,index)`.
- `addBack(e)`: ajoute un élément à la fin de la liste.
 - Peut être définie avec `add(e,index)`.
- `remove(index)`: supprime l'élément à la position `index` et le retourne.
- `removeFront()`: supprime l'élément au début de la liste et le retourne.
 - Peut être définie avec `remove(index)`.
- `removeBack()`: supprime l'élément à la fin de la liste et le retourne.
 - Peut être définie avec `remove(index)`.

```
21         @abc.abstractmethod
22         def insert_at(self, item, index):
23             raise NotImplementedError
24
25         @abc.abstractmethod
26         def add_front(self, item):
27             raise NotImplementedError
28
29         @abc.abstractmethod
30         def add_back(self, item):
31             raise NotImplementedError
32
33         @abc.abstractmethod
34         def remove(self, index):
35             raise NotImplementedError
36
37         @abc.abstractmethod
38         def remove_front(self):
39             raise NotImplementedError
```

Opérations sur une ADT liste (2/2):

Interface IList

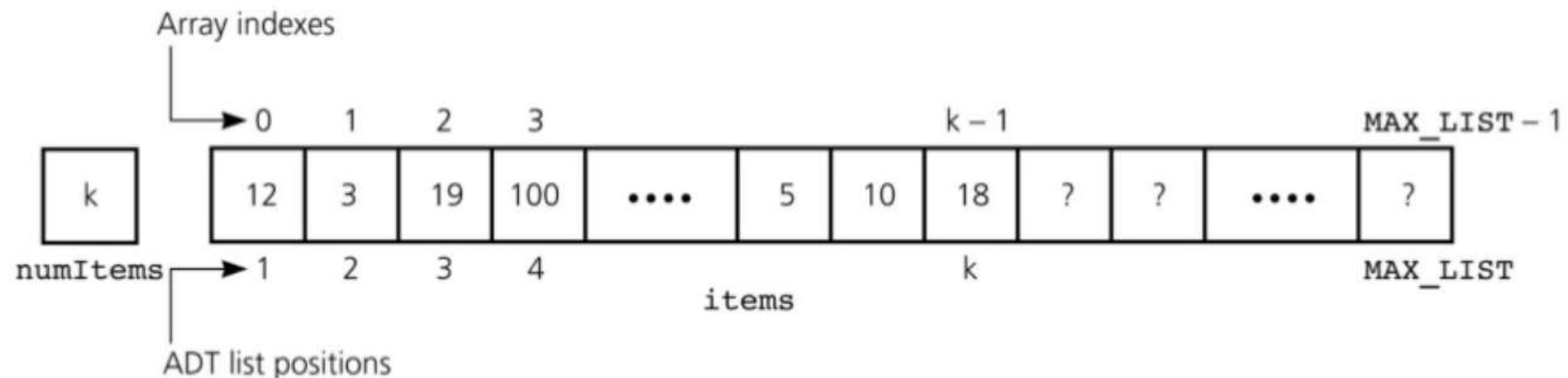
- `get(index)` : retourne l'élément à la position `index`.
- `put(index,e)`: remplace l'élément à la position `index` avec le nouveau élément `e`.
- `find(e)`: retourne la position de l'élément `e` dans la liste, sinon -1
- `concat(other)` : ajoute les éléments de la liste *other* à la liste courante.
 - Peut être définie avec `addBack(e)`.
- `length()` : retourne le nombre d'éléments dans la liste.
- `isEmpty()`: vérifie si la liste est vide.
 - Peut être définie avec `length()`.

```
41 @abc.abstractmethod
42 def remove_back(self):
43     raise NotImplementedError
44
45 @abc.abstractmethod
46 def get(self, index):
47     raise NotImplementedError
48
49 @abc.abstractmethod
50 def put(self, item, index):
51     raise NotImplementedError
52
53 @abc.abstractmethod
54 def find(self, item):
55     raise NotImplementedError
56
57 @abc.abstractmethod
58 def concat(self, other):
59     raise NotImplementedError
```

Implémentation contiguë d'une liste (1/3) :

Le principe

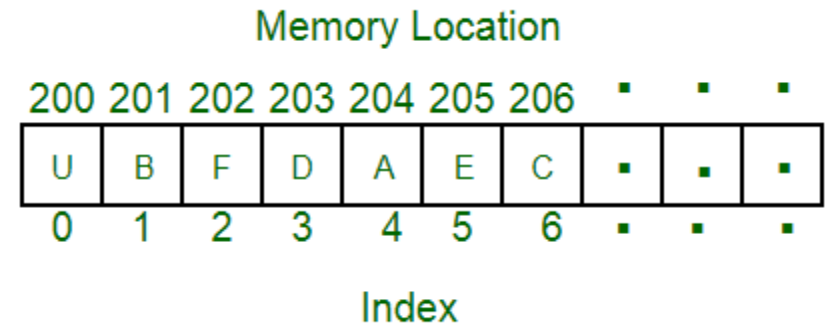
- Les éléments sont rangés les uns à côté des autres dans un tableau
 - La i ème case du tableau contient le i ème élément de la liste
 - Le rang est donc égal à la place ; ce sont des entiers
- Implémentation = structure (longueur, tableau)



Implémentation contiguë d'une liste (2/3) :

Le pour et le contre

- Avantages :
 - Simple à implémenter
 - Bon contrôle de la place
 - Nécessité de maintenir la longueur (inférieure à la taille du tableau !)
 - Accès rapide aux éléments
- Inconvénients :
 - Concaténation difficile
 - Modifications lentes, nécessitant souvent la recopie (insertion ou suppression d'un élément)
 - Convient mieux pour les listes non modifiables.



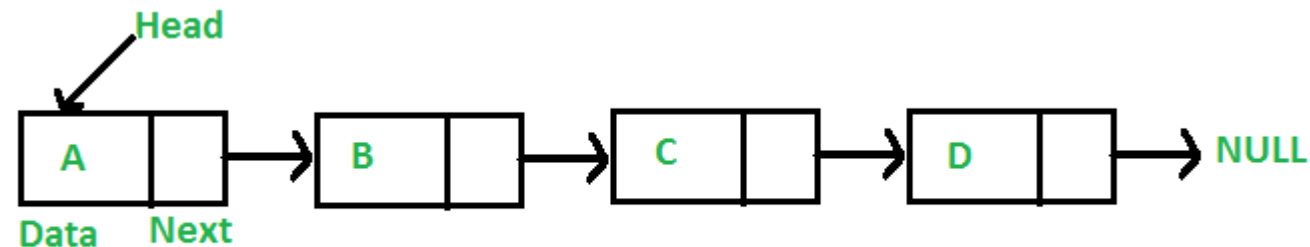
Implémentation contiguë d'une liste (2/3) :

Complexité asymptotique des opérations

Method	Time
size()	$O(1)$
isEmpty()	$O(1)$
get(i)	$O(1)$
set(i, e)	$O(1)$
add(i, e)	$O(n)$
remove(i)	$O(n)$

Implémentation chaînée d'une Liste (1/5) : Singly Linked List

- Les éléments ne sont pas rangés les uns à côté des autres
- La place d'un élément est l'adresse d'une structure qui contient l'élément ainsi que la place de l'élément suivant
- Utilisation de référence pour chaîner entre eux les éléments successifs
- Implémentation :
 - Chaque cellule est une structure comportant une référence vers la cellule suivante (type récursif).
 - La liste garde uniquement la première cellule.
 - L'accès se fait en suivant les références.



Implémentation chaînée d'une Liste (2/5) :

Le pour et le contre

- Avantages :
 - Rapidité des opérations, car pas de recopie des données ; adjonction, effacement, fusion de listes faciles ;
 - Gestion dynamique de la mémoire ;
 - Implémentation se prêtant bien à la récursivité (liste = tête de liste suivie du reste de la liste).
- Inconvénients :
 - Gestion dynamique de la mémoire (allocations et libérations explicites) ; un peu plus gourmand en mémoire ;
 - Pas d'accès direct et accès séquentiel dans une seule direction dans le cas d'un simple chaînage.

Implémentation chaînée d'une Liste (3/5) :

Ordre de grandeur des opérations

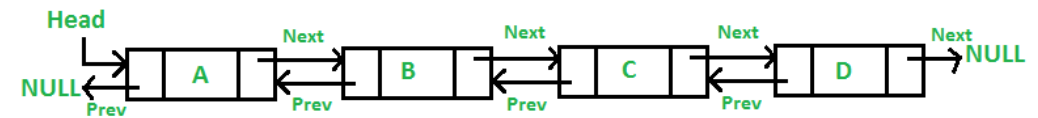
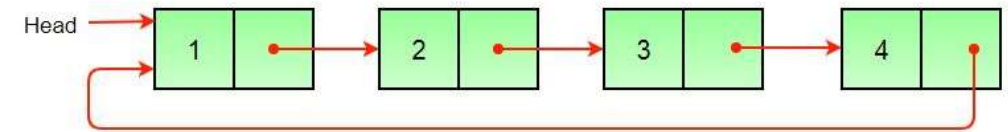
- Liste simplement chaînée avec une seule tête :

Opération	Représentation contiguë	Représentation chaînée
<i>liste_vide</i>	$O(1)$	$O(1)$
<i>ajout/suppression en tête</i>	$O(n)$	$O(1)$
<i>ajout/suppression générale</i>	$O(n)$	$O(n)$
<i>ajout/suppression en queue</i>	$O(1)$	$O(n)$
<i>accès</i>	$O(1)$	$O(n)$
<i>concaténation</i>	$O(n)$	$O(n)$

Implémentation chaînée d'une Liste (4/5) :

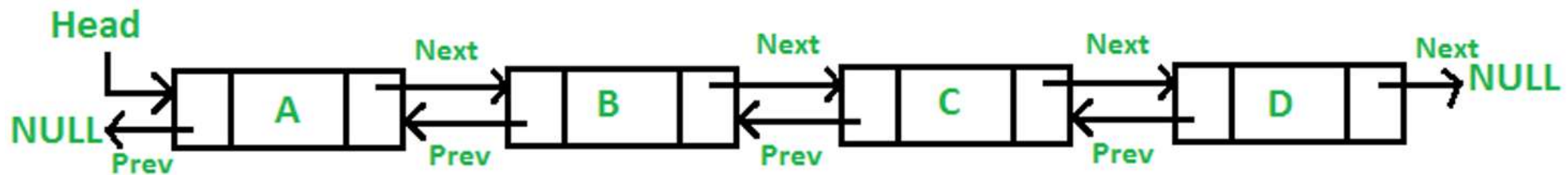
Variantes pour les implementations

- Liste chaînée circulaire
- Liste doublement chaînée
- Liste doublement chaînée circulaire
- Liste triée :
 - Les éléments sont munis d'une clé
 - Dans cette liste, il existe un ordre total sur les clés
 - L'ordre des enregistrements dans la liste respecte l'ordre sur les clés



Implémentation chaînée d'une Liste (5/5) : Doubly linked list

- Avantages
 - Faciliter le parcours de la liste dans les deux sens.
 - Utiliser un double chaînage ; chaque place repérant à la fois la place qui la précède et celle qui la suit.
- Inconvénients
 - Besoin de plus d'espace pour représenter les références supplémentaires
 - Plus d'opérations dans l'ajout et la suppression des éléments.



Applications des listes

- Structure de base pour les plusieurs ADT
 - Stack
 - Queue
 - Map
- Codage des polynômes
- Codage des matrices creuses
- Représentation d'un graphe (listes d'adjacences)

References

- <https://www.geeksforgeeks.org/introduction-to-data-structures-10-most-commonly-used-data-structures/>